

# Functions and arguments

Project: Python readiness portfolio

## 1. Prepare the purpose, prerequisites and materials

### Teach the learning cycle

Primary target: Distinguish a function definition from a function call

#### Session · Sequence and minutes · Resources

**Session:** Session 1 · Understand · 45 min

**Sequence and minutes:** 0–7: recall and prerequisite; 7–15: inspect the concrete case and predict; 15–30: teacher models the worked trace; 30–40: learners explain a step using the text and visual; 40–45: check the core idea.

**Resources:** Sections 1–6; record the brief recall in the workbook.

---

**Session:** Session 2 · Practise · 45 min

**Sequence and minutes:** 0–5: retrieve the procedure; 5–15: W2 completion practice; 15–23: explain and check with a partner; 23–30: each learner answers the misconception check; 30–40: targeted reteaching or a harder variation; 40–45: prepare the investigation.

**Resources:** Sections 7–10; use the matching workbook W2 and misconception check.

---

**Session:** Session 3 · Investigate and transfer · 45 min

**Sequence and minutes:** 0–5: retrieve the decision rule; 5–20: investigate or trace; 20–30: start the independent case; 30–38: connect the project; 38–43: act on feedback; 43–45: plan the return check.

**Resources:** Sections 9 and 11–14. Give extra time to finish the full task set; use Section 15 when the week includes a coding lab.

---

This is a starting allocation of 135 minutes, not a validated duration. Preserve all named topics. Schedule extra time for connected examples, the full investigation, independent cases and project building. Repeated copies of a case share one recorded attempt; they are not new evidence.

Case exposure check: this source example appears in ai-2/2, python-bridge/8. Reuse a familiar case for

recall or modelling; verify that the case used to assess transfer has not already received feedback.

## Retrieval answer guidance

### Lists and indexing

Without opening the old lesson, explain one key idea from Lists and indexing. Give an example and a mistake that the idea helps you avoid.

Look for: Distinguish a list index from the value stored at that position

Reference example, not the only acceptable wording: Length 3; indices 0, 1, 2; counts[1] is 7. The equal values occupy different positions, indices 0 and 2.

### Branching and boundary tests

Without opening the old lesson, explain one key idea from Branching and boundary tests. Give an example and a mistake that the idea helps you avoid.

Look for: Trace the selected branch in an if elif else chain

Reference example, not the only acceptable wording: if tests negative first; elif checks  $\leq 12$  only after that fails; else handles the rest. Unindented print runs after any selected branch, whose assignment gives label a value.

### Values variables and types

Without opening the old lesson, explain one key idea from Values variables and types. Give an example and a mistake that the idea helps you avoid.

Look for: Distinguish integer, floating-point, text and Boolean values

Reference example, not the only acceptable wording: 12 is int; 1.5 is float; "12" is str; False is bool; "False" is str. Quotes mark text even when the characters spell a Boolean name.

Teach definition, call, parameter and positional argument as distinct ideas. Keep inputs scalar and avoid mutable-list aliasing in this first function lesson. Reference: <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>. These functions print results; reusable return values and local state are taught in Week 9.

### Outcomes

Distinguish a function definition from a function call

Match positional arguments to parameters in order

Reuse one function and test that it uses the supplied values

### Preparation

Python 3 editor and functions.py

Parameter cards labelled capacity and used

Call cards and a table for expected and observed output

### Prerequisite answer

8. Subtraction is ordered: reversing these values would produce -8.

### End goal for the pictorial case

Trace functions and arguments and explain the changed program before running it.

This added worked case illustrates the week's main idea. Retain all original and connected topics; schedule its practice within the teaching cycle.

## 2. Explain and model the mechanism

A fictional supply cupboard records capacity and the number of occupied spaces. The remaining-space calculation is capacity minus used. Instead of copying the same instructions for every cupboard, define one function and call it with different values. A function makes a procedure reusable; its name alone does not guarantee that its instructions are correct.

Our input contract requires nonnegative integer counts with used no greater than capacity. The example assumes these conditions; it does not validate them. This week the function prints its result. Week 9 will show how to return a value for later calculations.

`def` introduces the definition. The function name is followed by parameter names in parentheses and a colon. Four-space indentation marks the body. Executing the definition creates the function but does not run its body. The two unindented calls then run it, producing 8 and 3 on separate lines. A blank line helps readability; indentation determines the block.

#### Term · Meaning

**Term:** Function

**Meaning:** A named block of reusable instructions that runs when called

---

**Term:** Definition

**Meaning:** Code that creates a function and specifies its body

---

**Term:** Call

**Meaning:** An instruction that runs a function with supplied arguments

---

**Term:** Parameter

**Meaning:** A name in a function definition that receives an argument

---

**Term:** Argument

**Meaning:** A value supplied when calling a function

---

## Arguments matched in order

### Call · Parameter values · Printed output

**Call:** show\_remaining(12, 4)

**Parameter values:** capacity 12; used 4

**Printed output:** 8

---

**Call:** show\_remaining(10, 7)

**Parameter values:** capacity 10; used 7

**Printed output:** 3

---

Each call runs the same body with the values supplied for that call.

The calls print 0 then 9. capacity receives 9 in both calls; used receives 9 then 0. Removing both calls leaves a definition with no printed output. Exit: show\_remaining(10, 7) should print 3 but the faulty version prints 8. Passing two cases does not establish correctness for all allowed inputs or validate the input contract.

## Model the reasoning aloud

Use the complete worked example below before asking learners to complete W2. Reveal a small part, invite a prediction, then explain the deciding step. These teacher notes contain solutions; do not reveal the independent case answer during modelling.

### Execute the definition · Create the function; no output yet

**Execute the definition:** Call with 12 and 4

**Create the function; no output yet:** Match parameters; subtract; print 8

---

**Execute the definition:** Call with 10 and 7

**Create the function; no output yet:** Match new values; subtract; print 3

---

**Execute the definition:** After the second call

**Create the function; no output yet:** Continue with the next statement

---

In `show_remaining(12, 4)`, 12 and 4 are arguments; capacity and used are parameters. Positional arguments match from left to right. `show_remaining(4, 12)` produces -8, showing why order matters; it also violates our written input contract. Use a fresh file with the definition before its calls.

**Predict the output of `show_remaining(9, 9)` followed by `show_remaining(9, 0)`. Name the parameter receiving each argument. What would be printed if both calls were removed?**

---

---

---

---

---

**Thinking move · Teacher prompt**

**Thinking move:** Plan

**Teacher prompt:** What is given? What is the question? Which procedure fits these facts?

---

**Thinking move:** Do

**Teacher prompt:** Point to each input and intermediate step in the worked trace. Name the rule that allows the next step.

---

**Thinking move:** Check

**Teacher prompt:** Does the output answer the question? Which assumption or limitation prevents a larger claim?

---

Use the diagram and verbal explanation together. Ask learners to match a sentence to a specific visual step; do not assign learners fixed visual or auditory learning styles.

## A pictorial worked case: Functions and arguments

Match arguments to parameters. A function prints the difference between capacity and reserved spaces.

### The facts we will use

Python 3: `def show_free(capacity,reserved): print(capacity-reserved)`. Calls: `show_free(11,3)`; `show_free(3,11)`.

AI ACADEMY / PYTHON-BRIDGE/8

## Functions and arguments

### Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



#### 1. Define without calling

The definition creates the function but prints nothing yet.



#### 2. Match first arguments

capacity=11 and reserved=3 give a printed 8.



#### 3. Trace reversed call

capacity=3 and reserved=11 give -8 under the same arithmetic.

### CHANGE ONE THING

Remove both calls and leave the definition only.

Explain the changed result using the matching lesson.

## Read the picture one step at a time

### Step · What happens and why

**Step:** 1. Define without calling

**What happens and why:** The definition creates the function but prints nothing yet.

---

**Step:** 2. Match first arguments

**What happens and why:** capacity=11 and reserved=3 give a printed 8.

---

**Step:** 3. Trace reversed call

**What happens and why:** capacity=3 and reserved=11 give -8 under the same arithmetic.

---

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

### Why this result follows

Positional arguments match parameter names in order. A function definition is not an execution of its body; separate calls supply separate input values.

### What this example does not establish

The arithmetic permits negative results; a real capacity contract may reject reserved>capacity instead of treating it as free space.

### Change one thing in the pictured case

Change: Remove both calls and leave the definition only.

Prediction: No output is printed.

Reason: The body runs only when the function is called.

### Ask while showing the picture

Cover the result. Ask learners to name the inputs and predict the next step. Uncover one step, connect the icon to the actual procedure, and explain its evidence. Then change the supplied condition and compare. The picture is a representation; it does not document execution of a real AI model.

## 3. Demonstrate and guide practice

### Time or stage · Teacher action

**Time or stage:** Session 1 start to 7 minutes

**Teacher action:** Retrieve ordered subtraction with the prerequisite.

---

**Time or stage:** 7 to 20 minutes

**Teacher action:** Read the contract and label the definition and calls for W1.

---

**Time or stage:** 20 to 33 minutes

**Teacher action:** Match argument cards to parameters and trace the worked calls.

---

**Time or stage:** 33 to 45 minutes

**Teacher action:** Complete W2 and the guided question before running the file.

---

**Time or stage:** Session 2 start to 8 minutes

**Teacher action:** Retrieve why a definition alone prints nothing.

---

**Time or stage:** 8 to 20 minutes

**Teacher action:** Run the fixed-number bug, complete W3 and rerun the earlier case after repair.

---

**Time or stage:** 20 to 35 minutes

**Teacher action:** Collect the independent W4 function and predictions before hints.

---

**Time or stage:** 35 to 45 minutes

**Teacher action:** Discuss W5 and use the exit to assess the limits of passing tests.

---

Use the worked example in Sections 3 and 6. Collect a prediction before revealing its result; ask the learner to point to each intermediate fact.

Use two labelled parameter spaces and place argument cards left to right. Permit an oral execution trace alongside code. Extension: add an if check inside `show_remaining` that prints Invalid when used exceeds capacity, otherwise prints the difference; test (4, 5) and (4, 4). This checks only that one constraint.

## Reduce help deliberately

Completion task: Using the correct definition, predict `show_remaining(8, 3)`, `show_remaining(8, 8)` and `show_remaining(8, 0)` in that order. Explain what happens if the file contains only the definition.

Teacher check: The calls print 5, 0 and 8 on separate lines. A definition alone creates the function but prints nothing because its body has not been called.

Start by identifying the given inputs together. Leave the intermediate steps blank. If needed, model one step, then withdraw that help on the next step. Move to a full trace only when the partial trace is understood. For partners, alternate explainer and checker; collect a response from every learner before discussion.

### Individual reasoning check

Use the worked case for Functions and arguments. Proposed explanation: “Writing def runs the body immediately”. Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

Expected correction: The definition creates the function; a call runs its body.

Collect explanations, not only yes/no votes. If a misunderstanding is common, re-model the relevant step for the class. If it affects a few learners, give a short targeted group explanation while others solve a more demanding variation.

### Pictorial practice question

Match every argument to a parameter for the two calls. Why do outputs differ, and what prints when both calls are removed?

### Reasoned teacher answer

The first call binds 11 to capacity and 3 to reserved, printing 8. The second reverses those bindings and prints -8. A definition with no calls prints nothing.

Changed-case reasoning: No output is printed. The body runs only when the function is called.

Collect each learner’s explanation before discussion. Use the first missing connection between input, deciding step and result to choose support. This question is worked-case practice, not fresh mastery evidence.

## 4. Run the investigation and diagnose misconceptions

Writing def runs the body immediately: The definition creates the function; a call runs its body.

Arguments and parameters are interchangeable labels: Parameters are names in the definition; arguments supply values at a call.

Swapping arguments never matters: Different parameter roles can give different results when values are swapped.

A correct first example proves the function is reusable: Fixed numbers can match one case while ignoring later arguments.

Use Section 9 for the evidence ledger. Add a boundary or missing-input case and record the expected response before checking. More successful repeats of one case do not establish wider coverage.

### Match the support to the first error

## What the response shows · Next teaching action

**What the response shows:** Missing or misread facts

**Next teaching action:** Read the case aloud, mark supplied versus unknown facts, and let the learner restate it.

---

**What the response shows:** Wrong procedure

**Next teaching action:** Return to the deciding step in the worked case. Contrast it with the proposed incorrect explanation.

---

**What the response shows:** Arithmetic or implementation error

**Next teaching action:** Trace intermediate values or lines; preserve the first mismatch and retry that step.

---

**What the response shows:** Correct result without a reason

**Next teaching action:** Ask for a trace and a changed case before marking independent understanding.

---

**What the response shows:** An unsupported wider claim

**Next teaching action:** Ask which supplied evidence supports the claim and which further observation would be needed.

---

Offer an oral explanation, drawing, enlarged visual or fewer examples when that removes a reading/writing barrier. Keep the same reasoning goal. Stretch secure learners by changing a boundary or assumption and asking them to defend the effect, not simply by assigning more of the same questions.

## 5. Assess and explain the answers

### Guided and exit questions

The calls print 0 then 9. capacity receives 9 in both calls; used receives 9 then 0. Removing both calls leaves a definition with no printed output. Exit: `show_remaining(10, 7)` should print 3 but the faulty version prints 8. Passing two cases does not establish correctness for all allowed inputs or validate the input contract.

### W1 Label the parts

In `def show_remaining(capacity, used)`, identify the function name and parameters. In `show_remaining(15, 6)`, identify the arguments and their matching parameters.

Name `show_remaining`; parameters `capacity` and `used`. Arguments 15 and 6 match `capacity = 15` and `used = 6`. Expected printed result 9.

## W2 Follow the execution order

Using the correct definition, predict `show_remaining(8, 3)`, `show_remaining(8, 8)` and `show_remaining(8, 0)` in that order. Explain what happens if the file contains only the definition.

The calls print 5, 0 and 8 on separate lines. A definition alone creates the function but prints nothing because its body has not been called.

## W3 Repair ignored arguments

The faulty body calculates  $12 - 4$  regardless of the call. State the output and expected output for `show_remaining(11, 2)`. Repair the expression and name an earlier call to rerun.

Faulty output 8; expected 9. Use `capacity - used`. Rerun `show_remaining(12, 4)`, expecting 8, as well as the revealing call.

## W4 Create a new function independently

Each sealed box contains 6 counters. Define `show_stock(boxes, loose)` to print the total counters using its two parameters. Assume nonnegative integer arguments. Call it with (3, 2), (0, 2) and (3, 0). Predict each output and explain why `show_stock(2, 3)` is a different case.

The body prints `boxes * 6 + loose`. Outputs are 20, 2 and 18. `show_stock(2, 3)` gives 15 because `boxes` and `loose` have different roles. Define once and use three calls; do not hard-code one answer.

## W5 State the input assumption

The correct subtraction function prints -1 for `show_remaining(4, 5)`. Does successful execution make this a valid cupboard record? Explain what the function assumes and what needs checking.

No. `used` exceeds `capacity` and violates the contract. The function calculates with supplied values but does not check the contract. A separate validation check or guarded function is needed to reject invalid records.

## Independent assessment and fresh retry

W4 must define one function, use both parameters, call it three times and predict 20, 2, 18; the swapped case must give 15. After support, retry `show_tiles(packs, singles)` with 4 tiles per pack: calls (2, 3), (0, 3), (2, 0) print 11, 3, 8; swapped (3, 2) prints 14. Record assistance and collect new predictions before execution.

## Assess independent transfer

Selected case: Predict both returned and printed values, showing which argument supplies first and second. What is displayed if the program contains only the function definition and no calls? Why does swapping these arguments change the result?

```
def difference(first, second):  
    return first - second  
  
print(difference(11, 4))  
  
print(difference(4, 11))
```

## Selected case answer

The first call returns and prints 7; the second returns and prints -7. Positional arguments fill first then second, and subtraction is ordered. A definition alone displays nothing and does not execute its body. The same function uses the supplied values for each call.

### Dimension · Evidence required

**Dimension:** Explanation

**Evidence required:** Names the applicable idea or procedure and ties it to the supplied facts.

---

**Dimension:** Trace and result

**Evidence required:** Preserves the given inputs and shows the relevant intermediate steps; distinguishes a prediction from an observed execution.

---

**Dimension:** Limits

**Evidence required:** States the assumptions, missing evidence or conditions under which the result would change.

---

Record each dimension as not yet, with support, or independent. Accept a valid alternative procedure with a defensible trace. Do not penalise an honestly recorded model failure if the learner correctly explains its evidence and limits.

## Feedback that the learner uses

Name the earliest unsupported step and give one action to repair it. Have the learner make that repair now and explain why. Then assess the relevant skill on a fresh, fully specified case. Confirm it was not previously practised with feedback; solve and check the new case before assigning it. A follow-up design prompt is a starting brief, not automatically an equivalent test.

## 6. Connect the project, support and next lesson

Add a labelled functions and arguments evidence sheet to Python readiness portfolio. Show how you can

distinguish a function definition from a function call. Use the supplied fictional case and keep its inputs, intermediate steps, calculation or prediction, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Use two labelled parameter spaces and place argument cards left to right. Permit an oral execution trace alongside code. Extension: add an if check inside `show_remaining` that prints Invalid when used exceeds capacity, otherwise prints the difference; test (4, 5) and (4, 4). This checks only that one constraint.

Week 9 returns results to the caller and explains local variables and fresh state on each call.

Retain the project ledger and assess a changed case after support. Check the relevant prerequisite before carrying this component forward.

## Plan the delayed return

Suggested return points in this level: python-bridge/9, python-bridge/11. Use actual classroom dates, adjusting for holidays and gaps. One, three and six teaching weeks are scheduling choices, not a claimed optimal spacing.

At a delayed check, ask for the idea from memory and a changed application. Compare with the earlier independent response and record support used. For a new level, revisit the previous capstone before checking the new prerequisites.

### Record · Teacher evidence

**Record:** Learner code and date

**Teacher evidence:** \_\_\_\_\_  
\_\_\_\_\_

**Record:** Concept and case used; prior exposure

**Teacher evidence:** \_\_\_\_\_  
\_\_\_\_\_

**Record:** Explanation / trace / limits

**Teacher evidence:** \_\_\_\_\_  
\_\_\_\_\_

**Record:** Support and first repair

**Teacher evidence:** \_\_\_\_\_  
\_\_\_\_\_

**Record:** Changed case and delayed outcome

**Teacher evidence:** \_\_\_\_\_  
\_\_\_\_\_

**Record:** Next teaching action

**Teacher evidence:** \_\_\_\_\_

\_\_\_\_\_

Use the evidence to choose reteaching or the next step. A completed page, confidence rating or attractive project does not by itself demonstrate understanding.

### **Project evidence from this case**

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted outputs and the changed-case explanation. Use actual Python 3 results to check the prediction after reasoning first.